

Universidad Nacional Toribio Rodríguez de Mendoza de
Amazonas

Facultad de Ingeniería de Sistemas y Mecánica Eléctrica

Material universitario

Comunicación asíncrona con AJAX, Fetch API, Promesas y JSON en PHP

Aplicación en un MVC desde cero con PHP puro, ORM, RBAC y middlewares

Curso: Desarrollo de Aplicaciones Web II - V ciclo

Docente: Mg. Eder Nicanor Figueroa Piscoya

Índice

1. Presentación del material	3
2. Resultados de aprendizaje	3
3. Competencias y capacidades	4
4. Fundamentos: de la web tradicional a AJAX	4
4.1. Ejemplo conceptual	5
5. Fetch API y promesas	5
5.1. Fetch básico con then/catch	6
5.2. Fetch con async/await	6
5.3. ¿Qué pasa si no uso async/await correctamente?	7
6. JSON en PHP: generar y consumir datos	7
6.1. Endpoint JSON en el controlador MVC	7
6.2. Lectura de JSON enviado por POST	8
7. Integración con la vista: recarga parcial de tabla	9
7.1. Vista HTML base	10
8. Búsqueda en tiempo real sin refrescar la página	10
8.1. Endpoint de búsqueda	11
8.2. Búsqueda con debounce	11
9. Nivel intermedio: estados de carga, errores y experiencia de usuario	12
10. Nivel avanzado: cancelación de peticiones con AbortController	13
11. AJAX dentro del MVC del curso: decisiones arquitectónicas	14
11.1. Opción 1: endpoint JSON	14
11.2. Opción 2: HTML parcial	14
11.3. Opción 3: API REST separada	14
12. Seguridad en comunicación asíncrona	15
12.1. RBAC y middlewares	15
12.2. CSRF	16
12.3. XSS	16

13. Debate crítico: ¿qué pasaría si no aplicamos comunicación asíncrona?	16
13.1. Preguntas para debate individual	17
13.2. Actividad grupal de pensamiento crítico	17
14. Laboratorio guiado de 120 minutos	18
14.1. Propósito	18
14.2. Materiales	18
14.3. Procedimiento	18
14.4. Producto esperado	19
15. Lista de cotejo para evaluación	19
16. Caso integrador propuesto	19
17. Errores frecuentes y cómo corregirlos	20
18. Síntesis final	21
Referencias bibliográficas	22

1. Presentación del material

Este material desarrolla la comunicación asíncrona en aplicaciones web modernas, desde los fundamentos de AJAX hasta el uso de *Fetch API*, promesas, `async/await`, JSON en PHP, recarga parcial de datos y búsqueda en tiempo real sin refrescar la página. El enfoque no es solamente técnico: se busca que el estudiante comprenda por qué esta capacidad cambió la forma de construir sistemas web, cómo impacta en proyectos reales y qué riesgos aparecen cuando se implementa sin criterios de arquitectura, rendimiento y seguridad.

La sesión está contextualizada al modelo MVC desarrollado en el curso, construido desde cero con PHP puro. El proyecto de referencia utiliza un `.htaccess` para redirigir las peticiones hacia `index.php`, un *front controller* que interpreta la URL, clases `Controller`, `Views`, controladores como `HomeController` y `UsersController`, vistas por carpetas, layouts reutilizables y métodos que pueden devolver HTML o JSON. Esta base permite convertir progresivamente una página tradicional renderizada desde el servidor en una interfaz dinámica que consume endpoints JSON sin abandonar el patrón MVC del curso (Figueroa, 2026).

En el desarrollo profesional actual, la comunicación asíncrona no debe verse como un adorno visual. Es una decisión arquitectónica que afecta experiencia de usuario, tiempos de respuesta percibidos, consumo de ancho de banda, separación cliente-servidor, pruebas, seguridad, trazabilidad y mantenimiento. La documentación oficial de MDN presenta *Fetch API* como una interfaz moderna para realizar solicitudes HTTP desde JavaScript, basada en promesas y orientada a un modelo más flexible que `XMLHttpRequest` (Mozilla Developer Network, 2025a). La especificación WHATWG, por su parte, define Fetch como el modelo estándar sobre el que se articulan solicitudes, respuestas, cabeceras, CORS y otros comportamientos de red en la plataforma web (WHATWG, 2025).

2. Resultados de aprendizaje

Al finalizar la semana, el estudiante será capaz de:

1. Explicar la diferencia entre navegación web tradicional, AJAX, `XMLHttpRequest` y *Fetch API*.
2. Implementar solicitudes GET y POST con `fetch()` dentro de una vista MVC creada en PHP puro.
3. Procesar respuestas JSON en JavaScript y generar respuestas JSON válidas desde un controlador PHP.

4. Aplicar promesas y `async/await` para controlar flujos asíncronos, errores y estados de carga.
5. Diseñar endpoints JSON respetando la separación de responsabilidades del patrón MVC, el ORM, RBAC y middlewares.
6. Implementar recarga parcial de tablas y búsqueda en tiempo real sin refrescar toda la página.
7. Argumentar, de forma crítica, los beneficios y riesgos de usar comunicación asíncrona en sistemas reales.

3. Competencias y capacidades

Dimensión	Capacidad esperada
Técnica	Construye endpoints JSON en PHP y los consume desde JavaScript usando <code>fetch()</code> , promesas y <code>async/await</code> .
Arquitectónica	Integra AJAX sin romper el MVC, separando rutas, controladores, modelos, vistas y middlewares.
Seguridad	Reconoce amenazas como XSS, CSRF, exposición de datos y ausencia de autorización en endpoints asíncronos.
Rendimiento	Evalúa impacto de recarga parcial, búsqueda en tiempo real, cancelación de peticiones y <i>debouncing</i> .
Pensamiento crítico	Debate cuándo conviene usar HTML parcial, JSON, SPA, htmx, Turbo o una API REST completa.

4. Fundamentos: de la web tradicional a AJAX

En una aplicación web tradicional, el usuario realiza una acción, el navegador envía una solicitud HTTP al servidor y el servidor responde con una página HTML completa. Este modelo es simple, fácil de entender y sigue siendo válido para muchos sistemas. Sin embargo, cuando toda interacción exige recargar la página completa, aparecen limitaciones: la experiencia se vuelve lenta, se pierde el estado de algunos componentes, se descargan nuevamente elementos repetidos y se incrementa la carga percibida.

AJAX, acrónimo de *Asynchronous JavaScript and XML*, popularizó la idea de intercambiar datos con el servidor de manera asíncrona para actualizar solo una parte de la página. Aunque el nombre incluye XML, en la práctica moderna JSON se convirtió en el formato predominante por su compatibilidad natural con JavaScript y su menor complejidad en comparación con XML (Garrett, 2005; Mozilla Developer Network, 2025a).

El aporte fundamental de AJAX no fue únicamente una técnica, sino un cambio en la experiencia de interacción: el navegador dejó de depender exclusivamente de la recarga completa para presentar información nueva.

4.1. Ejemplo conceptual

Sin AJAX, un listado de usuarios se actualiza así:

1. El usuario abre `/users/listar`.
2. El controlador consulta el modelo.
3. El servidor renderiza `Views/Users/listado.php`.
4. El navegador reemplaza toda la página.

Con AJAX o Fetch, el flujo puede cambiar:

1. El usuario escribe en un buscador.
2. JavaScript envía una solicitud a `/users/buscar?q=ana`.
3. El controlador consulta el modelo y devuelve JSON.
4. JavaScript actualiza solo el `tbody` de la tabla.

La diferencia no es superficial. En un sistema comercial, académico o ERP, esta técnica permite consultar productos, filtrar clientes, validar documentos, calcular totales y actualizar estados sin interrumpir la tarea principal del usuario.

5. Fetch API y promesas

Fetch API permite realizar solicitudes HTTP desde JavaScript mediante la función `fetch()`. Su diseño se basa en promesas, por lo que una solicitud no bloquea la ejecución completa del navegador mientras espera la respuesta del servidor (Mozilla Developer Network, 2025a). Una promesa representa un valor que puede estar disponible ahora, en el futuro o nunca, y puede encontrarse en estado pendiente, cumplido o rechazado (Mozilla Developer Network, 2025b).

5.1. Fetch básico con then/catch

Listing 1: Solicitud GET básica con fetch y promesas.

```
fetch(BASE_URL + '/users/apiListar')
  .then(response => {
    if (!response.ok) {
      throw new Error('Error HTTP: ' + response.status);
    }
    return response.json();
  })
  .then(data => {
    console.log('Usuarios recibidos:', data);
  })
  .catch(error => {
    console.error('Fallo la solicitud:', error);
  });
```

Este ejemplo muestra tres ideas importantes. Primero, `fetch()` devuelve una promesa. Segundo, convertir una respuesta a JSON también es una operación asíncrona. Tercero, no basta con asumir que todo salió bien; se debe verificar `response.ok` y capturar errores.

5.2. Fetch con async/await

`async/await` permite escribir código asíncrono con una estructura similar al código secuencial. No elimina las promesas; las utiliza de una forma más legible. Una función marcada como `async` devuelve siempre una promesa, y `await` pausa la ejecución de esa función hasta que la promesa se resuelva o sea rechazada (Mozilla Developer Network, 2025b).

Listing 2: Solicitud GET usando `async/await`.

```
async function cargarUsuarios() {
  try {
    const response = await fetch(BASE_URL + '/users/apiListar');

    if (!response.ok) {
      throw new Error('Error HTTP: ' + response.status);
    }

    const usuarios = await response.json();
    renderizarTabla(usuarios);
  } catch (error) {
    mostrarMensaje('No se pudieron cargar los usuarios.');
```

```
        console.error(error);  
    }  
}
```

5.3. ¿Qué pasa si no uso async/await correctamente?

Si el estudiante llama a `fetch()` y espera obtener los datos inmediatamente, cometerá un error frecuente. La solicitud no entrega el resultado de manera instantánea; devuelve una promesa. Por ello, el siguiente código no funciona como muchos principiantes esperan:

Listing 3: Error frecuente al tratar fetch como si fuera síncrono.

```
const usuarios = fetch(BASE_URL + '/users/apiListar');  
console.log(usuarios); // Muestra una Promise, no el arreglo de usuarios.
```

Este punto es clave para el pensamiento algorítmico. En comunicación asíncrona, el orden de escritura del código no siempre coincide con el orden de finalización de las operaciones. Por eso se requiere comprender promesas, errores, estados de carga y sincronización de eventos.

6. JSON en PHP: generar y consumir datos

JSON es un formato de intercambio de datos ligero, ampliamente usado en aplicaciones web. En PHP, `json_encode()` convierte valores PHP a una cadena JSON, y `json_decode()` convierte una cadena JSON en arreglos u objetos PHP (PHP Group, 2025a). Para leer el cuerpo crudo de una solicitud, especialmente cuando se envía JSON mediante POST, puede usarse `php://input`, un flujo de solo lectura que permite acceder al contenido de la petición HTTP (PHP Group, 2025b).

6.1. Endpoint JSON en el controlador MVC

En el MVC del curso, el controlador `UserController` ya presenta métodos como `listar()` y `ver($id)`. Para trabajar con AJAX, se puede agregar un método que devuelva JSON en lugar de renderizar una vista completa.

Listing 4: Endpoint JSON en `UserController.php`.

```
class UserController extends Controller {  
  
    public function apiListar() {  
        header('Content-Type: application/json; charset=utf-8');  
  
        // Ejemplo usando el ORM trabajado en clase
```



```
$usuarios = $this->model
    ->select('id, nombre, correo')
    ->orderBy('id', 'DESC')
    ->get();

echo json_encode([
    'ok' => true,
    'data' => $usuarios
]);
}
```

Este método respeta la arquitectura del curso: la vista no consulta directamente la base de datos; el controlador coordina la petición; el modelo u ORM obtiene los datos; y la respuesta se entrega en formato JSON.

6.2. Lectura de JSON enviado por POST

Listing 5: Lectura de JSON desde php://input.

```
public function apiGuardar() {
    header('Content-Type: application/json; charset=utf-8');

    $input = file_get_contents('php://input');
    $body = json_decode($input, true);

    if (!$body || empty($body['correo']) || empty($body['password'])) {
        http_response_code(422);
        echo json_encode([
            'ok' => false,
            'message' => 'Datos incompletos.'
        ]);
        return;
    }

    $data = [
        'correo' => $body['correo'],
        'password' => password_hash($body['password'], PASSWORD_DEFAULT)
    ];

    $id = $this->model->insert($data);
}
```

```
    echo json_encode([
        'ok' => true,
        'message' => 'Usuario registrado correctamente.',
        'id' => $id
    ]);
}
```

7. Integración con la vista: recarga parcial de tabla

La recarga parcial consiste en actualizar solo una parte del DOM. En lugar de renderizar toda la página, se modifica un fragmento específico, por ejemplo el cuerpo de una tabla.

Listing 6: Renderizar tabla de usuarios desde JSON.

```
function renderizarTabla(respuesta) {
    const tbody = document.getElementById('tbodyUsuarios');
    tbody.innerHTML = '';

    const usuarios = respuesta.data || [];

    usuarios.forEach(user => {
        const tr = document.createElement('tr');

        const tdId = document.createElement('td');
        tdId.textContent = user.id;

        const tdNombre = document.createElement('td');
        tdNombre.textContent = user.nombre;

        const tdCorreo = document.createElement('td');
        tdCorreo.textContent = user.correo;

        tr.appendChild(tdId);
        tr.appendChild(tdNombre);
        tr.appendChild(tdCorreo);
        tbody.appendChild(tr);
    });
}
```

Es importante usar `textContent` cuando se insertan datos provenientes del servidor, porque reduce el riesgo de inyectar HTML malicioso. OWASP recomienda aplicar codifi-

cación de salida contextual para prevenir XSS y evitar la inserción directa de contenido no confiable mediante HTML sin control (OWASP Foundation, 2024a).

7.1. Vista HTML base

Listing 7: Fragmento de Views/Users/listado.php.

```
<h1>Usuarios</h1>

<input type="text" id="txtBuscar" placeholder="Buscar por nombre o correo">
<button id="btnRecargar">Recargar</button>

<table border="1">
  <thead>
    <tr>
      <th>ID</th>
      <th>Nombre</th>
      <th>Correo</th>
    </tr>
  </thead>
  <tbody id="tbodyUsuarios">
    <!-- Contenido cargado con Fetch -->
  </tbody>
</table>

<script>
  const BASE_URL = '<?= BASE_URL ?>';
</script>
<script src="<?= BASE_URL ?>/Assets/js/users.js"></script>
```

8. Búsqueda en tiempo real sin refrescar la página

La búsqueda en tiempo real es uno de los ejemplos más útiles para comprender la comunicación asíncrona. Cada vez que el usuario escribe, el navegador puede enviar una consulta al servidor y actualizar los resultados sin recargar la página. Sin embargo, si se envía una petición por cada tecla sin control, se pueden generar problemas de rendimiento, saturación del servidor y respuestas fuera de orden. Por eso se recomienda aplicar *debouncing*, que consiste en esperar un breve intervalo antes de ejecutar la búsqueda (Google Web.dev, 2024a).

8.1. Endpoint de búsqueda

Listing 8: Endpoint de búsqueda con parámetro q.

```
public function apiBuscar() {
    header('Content-Type: application/json; charset=utf-8');

    $q = $_GET['q'] ?? '';
    $q = trim($q);

    $usuarios = $this->model
        ->select('id, nombre, correo')
        ->where('estado', '=', 1)
        ->like('nombre', $q)
        ->orWhere('correo', 'LIKE', "%$q%")
        ->orderBy('id', 'DESC')
        ->get();

    echo json_encode([
        'ok' => true,
        'data' => $usuarios,
        'query' => $q
    ]);
}
```

En un proyecto real, este ejemplo debe mejorarse para agrupar correctamente condiciones AND/OR, usar consultas preparadas en el ORM y evitar que el `orWhere` rompa la lógica de filtros por estado o permisos. Esta observación es valiosa porque conecta el tema de Fetch con el tema anterior del curso: ORM, RBAC y middlewares.

8.2. Búsqueda con debounce

Listing 9: Búsqueda en tiempo real con debounce.

```
let timerBusqueda = null;

const inputBuscar = document.getElementById('txtBuscar');
inputBuscar.addEventListener('input', () => {
    clearTimeout(timerBusqueda);

    timerBusqueda = setTimeout(() => {
        buscarUsuarios(inputBuscar.value);
    }, 400);
});
```

```
});

async function buscarUsuarios(texto) {
  try {
    const url = BASE_URL + '/users/apiBuscar?q=' + encodeURIComponent(texto);
    const response = await fetch(url);

    if (!response.ok) {
      throw new Error('Error HTTP: ' + response.status);
    }

    const data = await response.json();
    renderizarTabla(data);
  } catch (error) {
    console.error(error);
  }
}
```

9. Nivel intermedio: estados de carga, errores y experiencia de usuario

Una implementación profesional no solo debe traer datos. También debe comunicar al usuario qué está ocurriendo. Los estados mínimos son: cargando, éxito, vacío y error. Esta lógica mejora la percepción de velocidad y reduce la incertidumbre del usuario. La métrica *Interaction to Next Paint* (INP), impulsada por Google como parte de Core Web Vitals, enfatiza la importancia de responder visualmente a las interacciones del usuario de forma oportuna (Google Web.dev, 2024b).

Listing 10: Estados de carga y error.

```
async function cargarUsuarios() {
  const mensaje = document.getElementById('mensaje');
  mensaje.textContent = 'Cargando usuarios...';

  try {
    const response = await fetch(BASE_URL + '/users/apiListar');
    if (!response.ok) throw new Error('Error HTTP: ' + response.status);

    const data = await response.json();

    if (!data.data || data.data.length === 0) {
```

```
    mensaje.textContent = 'No hay usuarios registrados.';
  } else {
    mensaje.textContent = '';
  }

  renderizarTabla(data);
} catch (error) {
  mensaje.textContent = 'Error al cargar datos. Intente nuevamente.';
}
}
```

10. Nivel avanzado: cancelación de peticiones con Abort-Controller

En una búsqueda en tiempo real, puede ocurrir que una solicitud lenta llegue después de otra más reciente. Esto produce resultados inconsistentes: el usuario escribió “ana”, luego “anar”, pero la respuesta de “ana” llegó al final y reemplazó los resultados correctos. Para evitarlo, se puede cancelar la solicitud anterior con **AbortController**, una interfaz que permite abortar operaciones asíncronas como solicitudes Fetch (Mozilla Developer Network, 2025c).

Listing 11: Cancelación de solicitudes anteriores con AbortController.

```
let controladorBusqueda = null;

async function buscarUsuarios(texto) {
  if (controladorBusqueda) {
    controladorBusqueda.abort();
  }

  controladorBusqueda = new AbortController();

  try {
    const url = BASE_URL + '/users/apiBuscar?q=' + encodeURIComponent(texto);
    const response = await fetch(url, {
      signal: controladorBusqueda.signal
    });

    if (!response.ok) throw new Error('Error HTTP: ' + response.status);

    const data = await response.json();
  }
}
```

```
renderizarTabla(data);  
} catch (error) {  
    if (error.name === 'AbortError') {  
        return; // La petición fue cancelada porque llegó una búsqueda nueva.  
    }  
    console.error(error);  
}  
}
```

Este patrón introduce un criterio profesional: no basta con hacer que funcione en condiciones ideales; hay que considerar latencia, redes lentas, respuestas desordenadas y acciones rápidas del usuario.

11. AJAX dentro del MVC del curso: decisiones arquitectónicas

En el MVC del curso, las rutas se resuelven desde `index.php`, el controlador se determina a partir de la URL, el método se invoca dinámicamente y las vistas se renderizan desde carpetas organizadas por controlador (Figueroa, 2026). Incorporar Fetch no significa abandonar ese diseño. Significa agregar rutas que puedan devolver JSON o fragmentos HTML según la necesidad.

11.1. Opción 1: endpoint JSON

La opción más flexible consiste en devolver JSON y renderizar la interfaz con JavaScript. Es útil cuando el cliente necesita manipular datos, actualizar tablas, gráficos, tarjetas o formularios.

11.2. Opción 2: HTML parcial

Otra alternativa consiste en que el controlador devuelva un fragmento HTML ya renderizado y JavaScript lo inserte en la página. Este enfoque reduce código JavaScript, pero acopla más la respuesta al HTML del servidor. Bibliotecas modernas como htmx y Turbo han retomado esta idea: mejorar la interactividad con menos JavaScript manual y más intercambio de HTML parcial (Big Sky Software, 2025; Hotwired, 2025).

11.3. Opción 3: API REST separada

Una API REST completa puede ser adecuada cuando el backend será consumido por múltiples clientes: web, móvil, escritorio o integraciones externas. Sin embargo, también

aumenta la complejidad del proyecto: versionamiento, autenticación por tokens, CORS, documentación y pruebas de contrato.

Enfoque	Ventaja	Riesgo	Uso recomendado
JSON + Fetch	Flexible y reusable	Más lógica en JavaScript	Tablas, filtros, formularios dinámicos
HTML parcial	Menos JavaScript	Mayor acoplamiento a vistas PHP	Recarga de fragmentos simples
API REST	Escalable a varios clientes	Mayor complejidad arquitectónica	Sistemas con app móvil o integraciones
SPA completa	Experiencia muy dinámica	Curva de aprendizaje alta	Sistemas grandes con frontend especializado

12. Seguridad en comunicación asíncrona

Los endpoints AJAX no son menos importantes que las rutas tradicionales. Un error frecuente es proteger la vista HTML, pero dejar sin middleware los endpoints JSON. Si un método como `/users/apiListar` devuelve información sensible, debe pasar por autenticación, autorización y validación de permisos.

12.1. RBAC y middlewares

Si el proyecto ya cuenta con RBAC y middlewares, cada endpoint asíncrono debe verificar:

1. Usuario autenticado.
2. Rol o permiso requerido.
3. Alcance de datos que puede consultar.
4. Método HTTP permitido.
5. Token CSRF para operaciones de escritura.

12.2. CSRF

CSRF ocurre cuando un sitio malicioso intenta ejecutar acciones en nombre de un usuario autenticado. OWASP recomienda aplicar tokens CSRF, validar origen cuando corresponda y no confiar solamente en cookies de sesión para operaciones sensibles (OWASP Foundation, 2024b). En formularios enviados con Fetch, el token puede incluirse en una cabecera.

Listing 12: Envío de token CSRF con Fetch.

```
async function guardarUsuario(datos) {  
  const token = document.querySelector('meta[name="csrf-token"]').content;  
  
  const response = await fetch(BASE_URL + '/users/apiGuardar', {  
    method: 'POST',  
    headers: {  
      'Content-Type': 'application/json',  
      'X-CSRF-TOKEN': token  
    },  
    body: JSON.stringify(datos)  
  });  
  
  return await response.json();  
}
```

12.3. XSS

XSS puede aparecer cuando se insertan datos no confiables en el DOM. Por ello, en los ejemplos se prefiere crear nodos y asignar `textContent`, en lugar de concatenar HTML con `innerHTML`. OWASP insiste en que la prevención de XSS depende de codificación contextual, validación, sanitización cuando corresponda y uso cuidadoso de APIs peligrosas del DOM (OWASP Foundation, 2024a).

13. Debate crítico: ¿qué pasaría si no aplicamos comunicación asíncrona?

Pregunta central de debate

Si un sistema web universitario, comercial o ERP sigue recargando toda la página para cada búsqueda, validación o cálculo, ¿sigue siendo aceptable para los usuarios actuales? ¿En qué casos sí y en qué casos no?

No aplicar comunicación asíncrona no significa que el sistema sea automáticamente incorrecto. Una aplicación tradicional puede ser más simple, más segura de razonar y más fácil de mantener en equipos pequeños. Sin embargo, en escenarios donde el usuario realiza muchas operaciones repetitivas, la ausencia de recarga parcial puede generar lentitud percibida, frustración, abandono de tareas y mayor consumo de recursos. En proyectos reales de estos años, los usuarios esperan búsquedas instantáneas, validaciones inmediatas, guardado sin interrupción y retroalimentación visual rápida (Google Web.dev, 2024b; Nielsen Norman Group, 2020).

13.1. Preguntas para debate individual

1. ¿En qué parte de un sistema académico conviene usar Fetch: matrícula, notas, horarios, usuarios o reportes?
2. ¿Qué riesgo aparece si una búsqueda en tiempo real consulta la base de datos por cada tecla sin *debounce*?
3. ¿Por qué un endpoint JSON debe pasar por RBAC aunque no renderice una vista?
4. ¿Es mejor devolver JSON o HTML parcial en el MVC del curso? Justifique con un caso.
5. ¿Qué fallaría en una red lenta si no se implementan estados de carga y control de errores?

13.2. Actividad grupal de pensamiento crítico

Formar equipos de cuatro estudiantes. Cada equipo debe elegir un módulo del proyecto final del curso y responder:

1. ¿Qué interacción podría mejorarse con Fetch?
2. ¿Qué endpoint JSON se necesitaría?
3. ¿Qué datos mínimos debe devolver?
4. ¿Qué middleware o permiso RBAC debe protegerlo?
5. ¿Qué riesgo de seguridad o rendimiento podría aparecer?

Cada equipo expone en cinco minutos y otro equipo formula una objeción técnica.

14. Laboratorio guiado de 120 minutos

14.1. Propósito

Implementar recarga parcial y búsqueda en tiempo real en el módulo de usuarios del MVC propio, usando PHP puro, ORM, Fetch API, promesas, JSON y control básico de errores.

14.2. Materiales

- Proyecto MVC del curso funcionando en XAMPP o entorno equivalente.
- Base de datos con tabla `users` o equivalente.
- ORM del curso con métodos `select()`, `where()`, `like()`, `orderBy()` y `get()`.
- Navegador con herramientas de desarrollador.
- Editor de código.

14.3. Procedimiento

Paso 1. Crear el método `apiListar()` en `UserController` y devolver JSON con `json_encode()`.

Paso 2. Probar el endpoint directamente desde el navegador o Postman.

Paso 3. Crear la estructura HTML de tabla en `Views/Users/listado.php`.

Paso 4. Crear el archivo `Assets/js/users.js`.

Paso 5. Implementar `cargarUsuarios()` con `fetch()` y `async/await`.

Paso 6. Implementar `renderizarTabla()` usando `createElement()` y `textContent`.

Paso 7. Agregar un campo de búsqueda `txtBuscar`.

Paso 8. Crear `apiBuscar()` en el controlador.

Paso 9. Implementar búsqueda con *debounce* de 400 ms.

Paso 10. Agregar manejo de errores y mensaje de estado.

Paso 11. Verificar que el endpoint esté protegido por middleware si requiere sesión.

Paso 12. Documentar capturas de consola, red y resultado final.

14.4. Producto esperado

El estudiante entregará una evidencia breve en PDF o Word con:

- Captura del endpoint JSON funcionando.
- Captura de la tabla cargada sin recargar la página.
- Captura de la búsqueda en tiempo real.
- Fragmentos de código del controlador y JavaScript.
- Respuesta a dos preguntas críticas del laboratorio.

15. Lista de cotejo para evaluación

N.	Criterio	Puntaje
1	Crea correctamente el endpoint JSON <code>apiListar()</code> en el controlador.	2
2	Usa <code>json_encode()</code> y cabecera <code>Content-Type: application/json</code> .	2
3	Consume el endpoint con <code>fetch()</code> y <code>async/await</code> .	2
4	Controla errores con <code>try/catch</code> y valida <code>response.ok</code> .	2
5	Renderiza la tabla mediante recarga parcial sin refrescar la página.	2
6	Implementa búsqueda en tiempo real con parámetro <code>q</code> .	2
7	Aplica <i>debounce</i> para evitar peticiones excesivas.	2
8	Inserta datos en el DOM de forma segura usando <code>textContent</code> .	2
9	Relaciona el endpoint con ORM, RBAC o middleware del MVC del curso.	2
10	Sustenta críticamente beneficios, riesgos y casos de uso.	2
Total		20

16. Caso integrador propuesto

Para el proyecto final del curso, cada equipo debe seleccionar un módulo y aplicar comunicación asíncrona de manera justificada. Algunos ejemplos:

Módulo	Interacción asíncrona	Justificación
Usuarios	Búsqueda por nombre, correo o rol	Reduce recargas y permite administración más rápida.
Productos	Consulta de stock mientras se registra una venta	Evita vender productos sin disponibilidad.
Clientes	Validación de DNI o correo existente	Previene duplicados antes de enviar el formulario completo.
Ventas	Cálculo dinámico de totales	Mejora experiencia y reduce errores manuales.
Reportes	Filtros por fecha sin recargar layout	Mantiene contexto y agiliza análisis.

17. Errores frecuentes y cómo corregirlos

Error	Causa probable	Corrección
Unexpected token <in JSON	El endpoint devuelve HTML o error PHP en vez de JSON.	Revisar ruta, controlador, cabecera y errores visibles.
<code>response.json()</code> falla	La respuesta no es JSON válido.	Probar endpoint en navegador o Postman.
Tabla no cambia	ID incorrecto en el DOM o JS no cargado.	Verificar consola y ruta del script.
Datos duplicados	No se limpia el <code>tbody</code> .	Usar <code>tbody.innerHTML = ""</code> antes de renderizar.
Búsqueda lenta	Se envían demasiadas peticiones.	Aplicar <i>debounce</i> y optimizar consulta SQL.
Endpoint accesible sin sesión	Falta middleware/RBAC.	Registrar ruta protegida o validar permisos.

18. Síntesis final

AJAX y Fetch no deben enseñarse como una receta aislada, sino como una evolución del diseño de sistemas web. En el contexto del MVC construido desde cero, este tema permite unir varios aprendizajes previos: rutas, controladores, modelos, ORM, vistas, layouts, middlewares y RBAC. El estudiante comprende que una misma aplicación puede entregar HTML completo, HTML parcial o JSON, dependiendo de la necesidad.

El nivel básico consiste en pedir datos y mostrarlos. El nivel intermedio exige manejar estados, errores y búsqueda eficiente. El nivel avanzado implica pensar en cancelación de peticiones, seguridad, permisos, consistencia, rendimiento y arquitectura. En proyectos reales, esta diferencia marca la distancia entre una interfaz que simplemente funciona y una aplicación web mantenible, segura y aceptable para usuarios actuales.

Referencias bibliográficas

- Big Sky Software. (2025). *htmx: High power tools for HTML*. GitHub. <https://github.com/bigskysoftware/htmx>
- Figueroa, E. (2026). *MVC desde cero* [Material de clase no publicado]. Universidad Nacional Toribio Rodríguez de Mendoza de Amazonas.
- Garrett, J. J. (2005). *Ajax: A new approach to web applications*. Adaptive Path. <https://adaptivepath.org/ideas/ajax-new-approach-web-applications/>
- Google Web.dev. (2024a). *Debounce your input handlers*. <https://web.dev/articles/debounce-your-input-handlers>
- Google Web.dev. (2024b). *Interaction to Next Paint (INP)*. <https://web.dev/articles/inp>
- Hotwired. (2025). *Turbo: The speed of a single-page web application without having to write any JavaScript*. GitHub. <https://github.com/hotwired/turbo>
- Marchetto, A., Tonella, P., & Ricca, F. (2008). State-based testing of Ajax web applications. *Proceedings of the 2008 International Conference on Software Testing, Verification, and Validation*, 121-130. <https://doi.org/10.1109/ICST.2008.52>
- Mesbah, A., & van Deursen, A. (2007). An architectural style for Ajax. *Proceedings of the Working IEEE/IFIP Conference on Software Architecture*. <https://doi.org/10.1109/WICSA.2007.38>
- Mozilla Developer Network. (2025a). *Using the Fetch API*. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch
- Mozilla Developer Network. (2025b). *Promise*. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise
- Mozilla Developer Network. (2025c). *AbortController*. MDN Web Docs. <https://developer.mozilla.org/en-US/docs/Web/API/AbortController>
- Nielsen Norman Group. (2020). *Response times: The 3 important limits*. <https://www.nngroup.com/articles/response-times-3-important-limits/>
- OWASP Foundation. (2024a). *Cross site scripting prevention cheat sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- OWASP Foundation. (2024b). *Cross-site request forgery prevention cheat sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html
- PHP Group. (2025a). *json_encode*. PHP Manual. <https://www.php.net/manual/en/function.json-encode.php>
- PHP Group. (2025b). *PHP input/output streams*. PHP Manual. <https://www.php.net/manual/en/wrappers.php.php>
- WHATWG. (2025). *Fetch standard*. <https://fetch.spec.whatwg.org/>